Assignment 6: Visitor Pattern

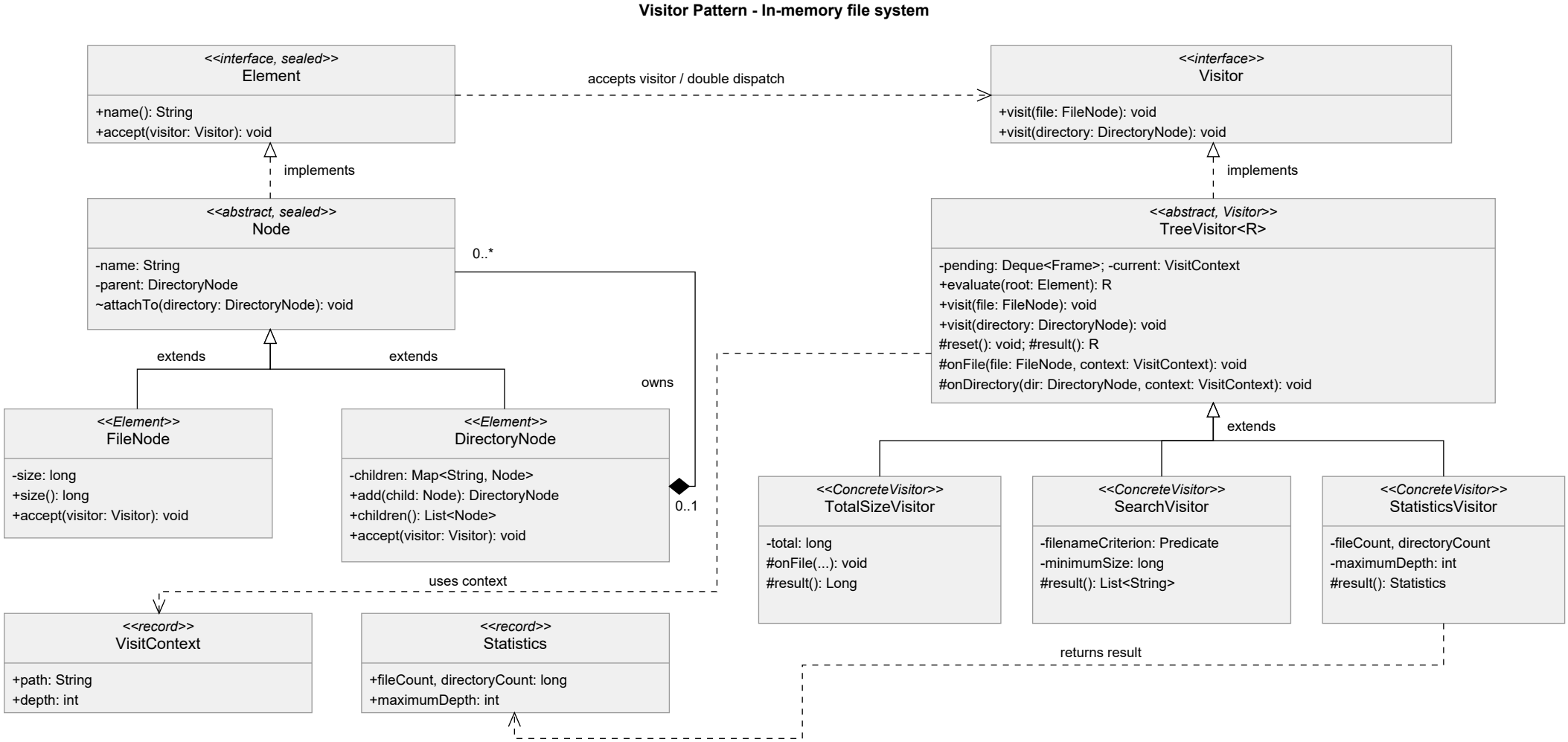
Group: SE-2538

1. Introduction

The selected domain is an in-memory hierarchical file system. `FileNode` stores a name and a non-negative byte size; `DirectoryNode` owns named child nodes. Three independent operations work on the same structure: total size, configurable filename-and-size search, and file/directory/depth statistics.

`Element` defines the common contract; both concrete elements dispatch to overloaded methods in `Visitor`. `TreeVisitor<R>` provides a single reusable traversal lifecycle. The model rejects negative sizes, duplicate sibling names, multiple parent ownership and cycles before invalid changes are applied.

2. UML class diagram



Dashed hollow triangles represent interface realization; solid hollow triangles represent inheritance. The filled diamond marks child ownership. Dashed open arrows show dependencies. Elements only dispatch; the common visitor owns traversal.

3. Clean Code principles

3.1 Replace type-checking chains with polymorphism

Each concrete element delegates to the matching overloaded visit method. This keeps behavior selection out of `instanceof`, `getClass()`, reflection and casting chains.

```
// FileNode.accept
public void accept(Visitor visitor) {
    Objects.requireNonNull(visitor, "visitor").visit(this);
}
```

First, the element's runtime type selects its `accept` method. Inside `FileNode.accept`, the static type of `this` selects `visit(FileNode)`; the call is dispatched on the visitor object. `TreeVisitor` then calls the operation's `onFile` hook. The directory overload works analogously.

3.2 Use intention-revealing names

```
// SearchVisitor fields
private final Predicate<String> filenameCriterion;
private final long minimumSize;
```

`filenameCriterion` states what is matched, while `minimumSize` identifies the inclusive byte threshold. `Statistics.maximumDepth` and `TotalSizeVisitor` similarly communicate their purpose without numeric operation codes.

3.3 Give each visitor a single responsibility

```
// TotalSizeVisitor: only file sizes contribute to the total.
protected void onFile(FileNode file, VisitContext context) {
    total = Math.addExact(total, file.size());
}
```

This visitor only accumulates bytes and explicitly rejects long overflow. Searching and statistics belong to separate visitors, traversal belongs to `TreeVisitor`, and console formatting belongs to `Demo`.

3.4 Keep methods small and remove duplicated rules

```
// StatisticsVisitor
private void includeDepth(VisitContext context) {
    maximumDepth = Math.max(maximumDepth, context.depth());
}
```

Both file and directory callbacks reuse `includeDepth`. This small method keeps one depth rule in one place and includes empty directories in the maximum.

3.5 Encapsulate state and protect valid mutations

```
// DirectoryNode
public List<Node> children() {
    return List.copyOf(children.values());
}
```

Clients receive an immutable snapshot, so they cannot edit the internal child map. All public additions pass through `DirectoryNode.add`, which checks sibling names, ownership and ancestor cycles. Names and file sizes are immutable; `null` and invalid values are rejected predictably.

4. Traversal, lifecycle and extension trade-offs

Visitors own traversal through `TreeVisitor.evaluate(root)`. An explicit stack performs DFS; elements only dispatch and every reachable node is visited once per operation. Keeping traversal in one place prevents duplicate walks and supports deep nesting without recursive calls. The root has depth 0; directory counts include a directory root and empty directories.

Paths are relative to the chosen root, use `/` and are sorted lexicographically. The root itself has path `.`, including a file root; its name is omitted from descendant paths. Search combines a filename predicate with `size >= minimumSize`.

Each run resets its result; `finally` clears the pending stack and context after success or failure. Instances are sequentially reusable, not thread-safe. Nested `evaluate` on the same instance is rejected; predicates must neither mutate the tree nor invoke visitors. Returned search lists and statistics are immutable.

The separate statistics commit adds `StatisticsVisitor`, its result record and tests without editing `FileNode`, `DirectoryNode`, their `accept` methods or the existing visitor contracts. A new element type is more expensive: it requires updating sealed permits, the Visitor overloads, traversal and operation semantics. Visitor favors adding operations to a stable set of element types.

5. Sample output

```
Root: campus
Total size: 3568 bytes
Search: suffix=.txt, minimum=300
  docs/architecture.txt
  docs/archive/old-notes.txt
Matches: 2
Files: 6
Directories: 6
Maximum depth: 3
```

The data is synthetic and defined in `Demo.sampleTree`. Running the JAR with `.png 1000` selects only `assets/logo.png`; totals and statistics remain unchanged.

6. Automated verification

`mvn test` and `mvn package` succeeded on Microsoft OpenJDK 17.0.20.1 and Maven 3.9.11. Surefire reports **25 tests, 0 failures, 0 errors and 0 skipped**. Both executable demo configurations were also run.

Test classes (test counts): `DemoTest` (2); `DispatchTest` (1); `SearchVisitorTest` (7); `StatisticsVisitorTest` (5); `TotalSizeVisitorTest` (4); `TreeIntegrityTest` (6).

T1-T8 cover double dispatch, nested totals, both search constraints, deterministic root-relative paths, counts/depth, empty/no-match cases, repeat runs and invalid mutations. Additional cases cover 2000 levels, overflow and predicate failures. Test fixtures and expected values are independent of the demo.

7. Conclusion

Visitor separates the file structure from independent operations while keeping invariants in the nodes. The shared traversal reduces duplication; the trade-off is a larger class structure and the need to update visitor contracts when new element types are introduced.

8. Link to the GitHub repository

https://github.com/bigstep832/visitor-pattern-assignment